

Advanced RAG Patterns

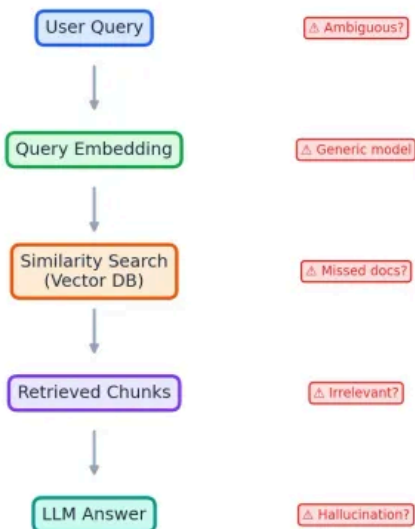
Comprehensive notes — building retrieval-augmented generation that holds up in production.

Most RAG systems work fine in demos and fall apart in production, for a predictable reason: basic RAG has five distinct points of failure, and most teams only discover them after shipping. These notes walk through every stage where things break and how to fix each one systematically — query transformation, intelligent routing, ColBERT-style retrieval, Chain-of-Note post-processing, and agentic correction loops.

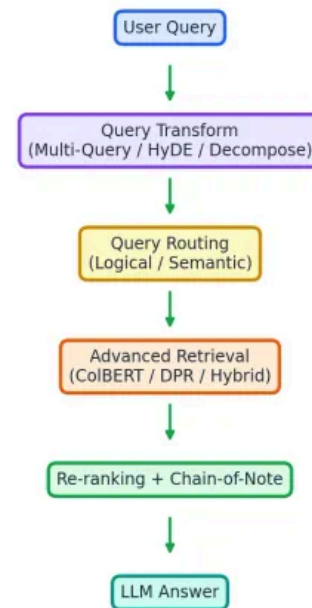
Part of the AI-PLATFORM reference. Technique explanations follow the source blog (see [reference/SOURCES.md](#)); each section adds worked **Observability** and **Security** use-case examples (included at explicit request). Diagrams in [images/](#). Export to PDF: open in a browser → Print → Save as PDF.

Basic vs Advanced RAG

Basic RAG Pipeline



Advanced RAG Pipeline



Contents

1. [Why Basic RAG Fails at Scale](#)
2. [Query Transformation](#)
3. [Query Routing](#)
4. [Advanced Indexing \(Multi-Rep + RAPTOR\)](#)
5. [Advanced Retrieval \(DPR + ColBERT\)](#)
6. [Re-ranking & Chain-of-Note](#)

- 7. [Agentic RAG: CRAG & Self-RAG](#)
- 8. [Evaluation Framework](#)
- 9. [Production Architecture Blueprint](#)
- 10. [Implementation Patterns](#)

1 · Why Basic RAG Fails at Scale

The basic pipeline has 5 failure points:



Stage	What goes wrong	Why it matters
User Query	Ambiguous, poorly worded	Garbage in, garbage out
Query Embedding	Generic model misrepresents intent	Wrong retrieval space entirely
Similarity Search	Semantically close ≠ answer-bearing	Top-K chunks miss the point
Retrieved Chunks	Irrelevant or missing context	LLM gets confused or hallucinates
LLM Answer	Ignores context, uses prior knowledge	Wrong answer delivered confidently

People build basic RAG and it works fine on demos. Then it hits production, user queries get weird, and retrieval starts returning chunks that are vaguely related but useless. The system looks smart but isn't.

Advanced RAG means intercepting at every one of those stages. Each arrow in the pipeline is an opportunity to add intelligence.

OBSERVABILITY EXAMPLE

Observability: A knowledge assistant over runbooks, dashboard docs, and past incident write-ups. "elevated tail latency after the last deploy" embeds with a generic model (wrong space), similarity-searches into generic latency chunks (semantically close $\neq$ answer-bearing), and the LLM confidently cites an irrelevant runbook (wrong answer, delivered confidently).

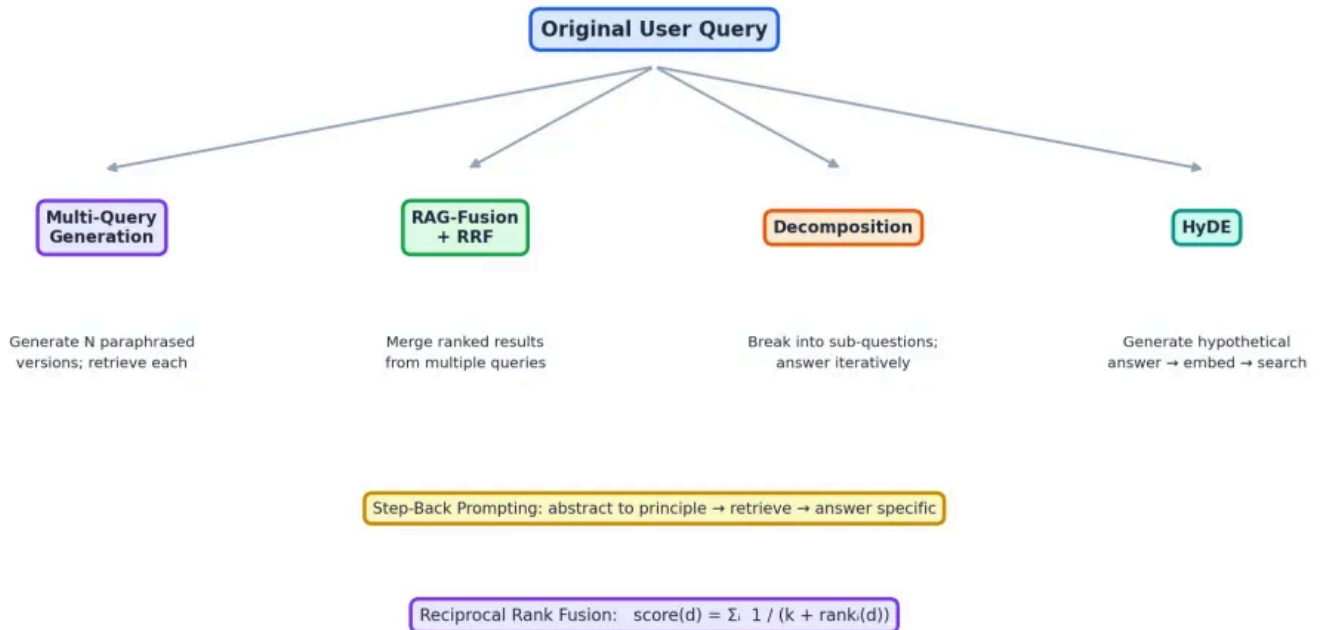
SECURITY EXAMPLE

Security: An assistant over threat intel, control catalogs, and prior findings. "Is CVE-relevant exposure present for service X?" needs multi-hop reasoning a single top-k lookup can't do, and returns stale advisories with no trust signal — every one of the 5 failure points is high-stakes in security.

2 · Query Transformation

The problem: users don't write good queries — they write what they're thinking, not what the vector database needs to hear. A single poorly-phrased query retrieves the wrong documents even if the answer is sitting right there in your corpus.

Query Transformation Techniques



Technique 1: Multi-Query Generation

Generate N paraphrased versions of the original query, retrieve separately for each, merge all results before ranking.

```
from langchain.retrievers.multi_query import MultiQueryRetriever

retriever = MultiQueryRetriever.from_llm(
    retriever=vectorstore.as_retriever(),
    llm=llm
)

docs = retriever.get_relevant_documents("What is the revenue of Company X in 2005?")
```

Typically **10–15% recall improvement** on ambiguous queries. Cost is extra LLM calls — worth it for any query where precision matters.

Technique 2: RAG-Fusion with Reciprocal Rank Fusion (RRF)

Generate multiple related queries, retrieve for each, then merge the ranked lists with RRF. Key insight: a document appearing consistently across multiple ranked lists is probably genuinely relevant.

```
def reciprocal_rank_fusion(results: list[list], k: int = 60) -> list:
    scores: dict = {}
    for ranked_list in results:
        for rank, doc in enumerate(ranked_list):
            doc_id = doc.page_content
            scores[doc_id] = scores.get(doc_id, 0) + 1 / (k + rank + 1)
    return sorted(scores.items(), key=lambda x: x[1], reverse=True)
```

$$\text{score}(d) = \sum_i 1 / (k + \text{rank}_i(d))$$

Expect **15–20% improvement** on queries with multiple valid angles. `k=60` is from the original paper and works well in practice.

Technique 3: Decomposition

Break a complex question into sequential sub-questions. Answer each, then feed those answers as context for the next.

```
# Use case: "Is Berlin warmer today than in 2005?"
# Sub-Q1: What is Berlin's current average temperature?
# Sub-Q2: What was Berlin's average temperature in 2005?
# Synthesis: Compare the two answers
```

The most underused technique. Analytical questions are almost never stated verbatim in your documents — a direct vector search returns general chunks, not the specific comparison. Decomposition forces retrieval of the atomic facts and reasoning over them.

Technique 4: HyDE (Hypothetical Document Embeddings)

Generate a hypothetical answer, embed *that*, then search for real documents similar to it. You're asking: what would a document that answers this look like?

```
from langchain.chains import HypotheticalDocumentEmbedder

hyde_embeddings = HypotheticalDocumentEmbedder.from_llm(
    llm=llm,
    base_embeddings=embeddings,
    custom_prompt=hyde_prompt
)
```

Powerful for domain-specific corpora where query language differs from document language. Tradeoff: hallucinated hypotheses can steer retrieval wrong — ground your prompts tightly.

Technique 5: Step-Back Prompting

Before searching, abstract the query to a higher-level principle. Retrieve for that abstraction, then answer the specific question with that broader context.

Original: "What happens to a gas when pressure increases at constant temperature?"

Step-Back: "What is Boyle's Law?"

Retrieve Boyle's Law docs → use them to answer the original question

Works well in scientific, legal, and technical domains where specific questions are really asking about general principles.

OBSERVABILITY EXAMPLE

Observability: "Why did checkout error rate spike at 2pm?" → **Decomposition:** (Sub-Q1) what changed near 2pm? (Sub-Q2) which dependency degraded? (Sub-Q3) matching past incidents? — then synthesize. **HyDE** bridges symptom-language ("users can't pay") to cause-language in runbooks ("payment gateway 5xx").

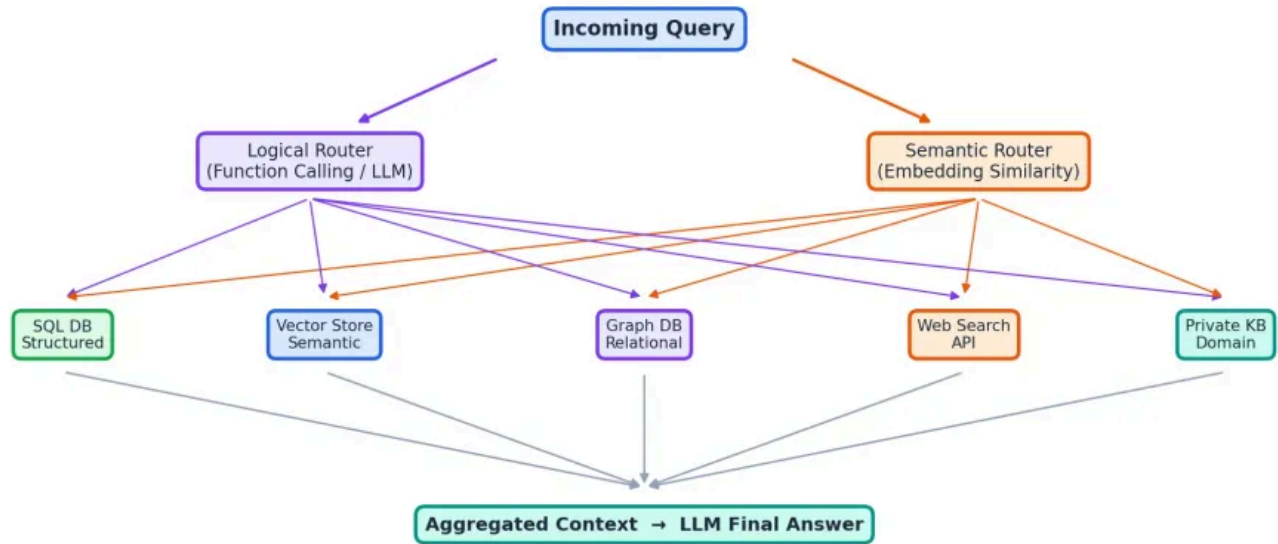
SECURITY EXAMPLE

Security: "Are we exposed to the new auth bypass?" → **Step-Back** to "authentication bypass classes and our controls," retrieve the control docs, then answer the specific advisory. **Multi-Query** covers synonym drift ("privilege escalation" / "priv-esc" / "elevation of privilege").

3 · Query Routing

The problem: real systems have a SQL database, a vector store, maybe a graph database, APIs, and a private KB. Sending every query to every source is expensive and noisy; sending it to the wrong source gives you nothing useful.

Query Routing Architecture



Logical Routing via Function Calling

Give the LLM a structured schema and let it decide which store to query. Works well when data sources have clear, non-overlapping domains.

```
from pydantic import BaseModel

class RouteQuery(BaseModel):
    datasource: Literal["python_docs", "js_docs", "golang_docs"]

structured_llm = llm.with_structured_output(RouteQuery)

def route(question: str) -> str:
    return structured_llm.invoke(question).datasource
```

Semantic Routing

Embed descriptions of each data source and route by embedding similarity — no LLM call, just a cosine check.

```

from langchain.utils.math import cosine_similarity

route_layer = {
    "sql_db": embed("Structured data, financial records, transactions"),
    "vector_store": embed("Semantic search over unstructured text"),
}

def semantic_route(query: str) -> str:
    q_embed = embed(query)
    scores = {k: cosine_similarity(q_embed, v) for k, v in route_layer.items()}
    return max(scores, key=scores.get)

```

Tool and API Routing

Data sources don't have to be static — route to live APIs where the LLM generates the parameters, the tool executes, and the output feeds back into the RAG context. This is where RAG starts blending into agentic systems (§7).

```

# LLM decides: "This needs real-time data"
# → calls weather_api(city="Berlin", date="today")
# → response becomes part of retrieved context

```

OBSERVABILITY EXAMPLE

Observability: "current p99 of the orders API" is a live metric → route to a metrics tool, not docs. "How do we page on-call?" → runbook index. "Explain this trace" → trace-analysis retriever. One router, three backends.

SECURITY EXAMPLE

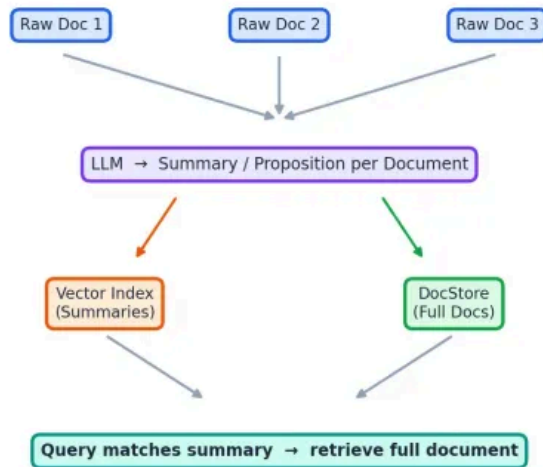
Security: "list open findings for account 123" → findings store (structured); "what's our retention policy?" → governance-docs index; "is this IP malicious?" → threat-intel API. Routing also enforces boundaries — queries only reach indexes the caller is authorized for.

4 · Advanced Indexing (Multi-Rep + RAPTOR)

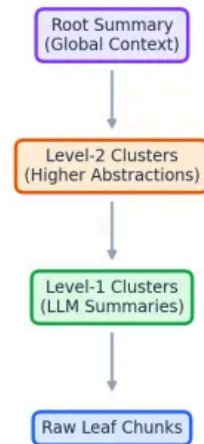
The problem: standard chunking is brutal. Split a 50-page doc into 512-token chunks, embed, and hope the right chunk surfaces. Flat indices don't understand document hierarchy, and the chunk that retrieves well often isn't the chunk that answers the question completely.

Advanced Indexing Strategies

Multi-Representation Indexing



RAPTOR: Recursive Abstractive Processing



At retrieval: search ALL levels simultaneously

Multi-Representation Indexing

Index small, return large. Use LLM-generated summaries or propositions for the vector index (they embed cleanly), but store the full documents separately. Retrieval hits the summary; generation gets the full document.

```
from langchain.storage import InMemoryByteStore
from langchain.retrievers.multi_vector import MultiVectorRetriever

summaries = [llm.invoke(f"Summarize: {doc}") for doc in docs]

store = InMemoryByteStore()
retriever = MultiVectorRetriever(
    vectorstore=vectorstore, # searches summaries
    byte_store=store,       # returns full docs
    id_key="doc_id",
)
retriever.docstore.mset(list(zip(doc_ids, docs)))
```

Solves a real problem: dense embeddings of long documents lose specificity because they average over too much content.

RAPTOR

Build a tree. Leaves are raw chunks; each level above clusters the nodes and has an LLM summarize each cluster, up to a single root node capturing the whole document's essence.

```
Level 0: Raw chunks (leaves)
Level 1: LLM summaries of chunk clusters
Level 2: Summaries of level-1 summaries
Root:    Global document summary
```

At query time, search all levels simultaneously — simple queries hit leaves, high-level conceptual queries hit the root, multi-hop questions find intermediate nodes. Cluster with Gaussian Mixture Models, summarize with an LLM. The paper reports **20%+ gains on multi-hop** questions ([arxiv 2401.18059](https://arxiv.org/abs/2401.18059)).

Query Construction for Structured Sources

If your vector store has metadata, use it — convert natural-language conditions into structured filters before searching.

```
# "What Python tutorials were published after 2023?"
metadata_filter = {"language": "python", "publish_date": {"$gt": "2023-01-01"}}
retriever = vectorstore.as_retriever(search_kwargs={"filter": metadata_filter})
```

A massive win on corpora with rich metadata. Construct the filter first, then search a smaller, cleaner subset.

OBSERVABILITY EXAMPLE

Observability: RAPTOR over a year of postmortems lets "what are our recurring reliability themes?" hit a high-level node while "what fixed the 2pm checkout outage?" hits a leaf. **Query construction** filters by `service` and `time-window` metadata so an incident query searches only the relevant, recent slice.

SECURITY EXAMPLE

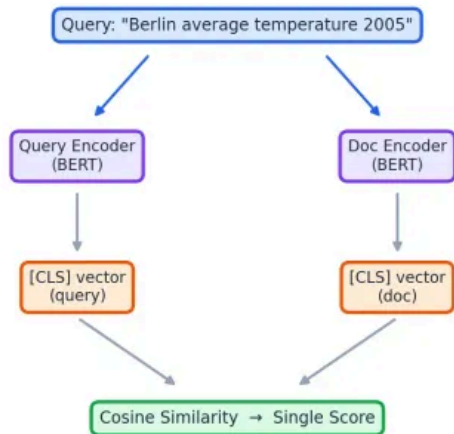
Security: Multi-representation indexes the intent of dense control clauses so semantic questions match legalistic text; **query construction** uses `sensitivity` / `owner` metadata to constrain retrieval to authorized subsets before searching — filtering, not post-filtering.

5 · Advanced Retrieval: DPR & ColBERT

The problem: general-purpose embedding models capture semantic *similarity*, not answer-bearing *relevance*. A passage about "temperature measurement instruments" is similar to "thermometer history" but may be useless for answering it. One vector collapses everything and loses nuance.

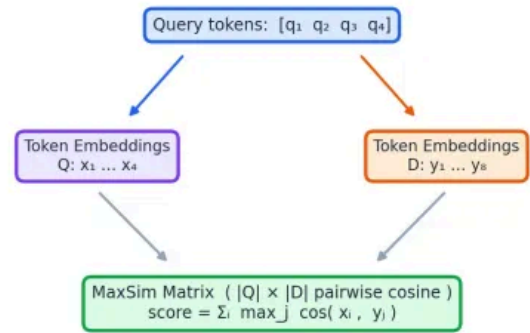
ColBERT Late Interaction vs Dense Retrieval

Dense Retrieval (Single Vector)



⚠ One vector loses token-level nuance

ColBERT: Late Interaction



Each query token finds its best matching document token

✓ >2x recall improvement over BM25

Dense Passage Retrieval (DPR)

Train separate encoders for queries and documents on actual Q&A pairs. The training objective is the key difference: contrastive loss on real question-answer pairs teaches the model what "answers this question" looks like, not just what's similar.

```
query encoder:    Q → q_vec
document encoder: D → d_vec
score = dot(q_vec, d_vec)
```

In-batch negatives during training force the model to distinguish answer-bearing passages from merely related ones.

ColBERT: Late Interaction

Standard retrieval compresses a document into one vector. ColBERT keeps one vector per token and compares at the token level.

```
Query tokens:    [q1, q2, q3, q4] → embeddings [x1, x2, x3, x4]
Document tokens: [d1..d8]          → embeddings [y1..y8]
```

```
MaxSim score =  $\sum_i \max_j \text{cosine}(x_i, y_j)$ 
```

For each query token, find the most similar document token and take that score; sum across query tokens. A query like "Gross Revenue of all products of Company X in 2005" gets scored on each key term independently rather than as a blended average.

```
from ragatouille import RAGPretrainedModel

RAG = RAGPretrainedModel.from_pretrained("colbert-ir/colbertv2.0")
RAG.index(collection=docs, index_name="my_index")
results = RAG.search(query="Gross revenue Company X 2005", k=10)
```

ColBERT achieves **over 2× recall improvement over BM25** on NaturalQuestions. Tradeoff: storage — you store token vectors per document rather than one. For most production use cases that's fine.

OBSERVABILITY EXAMPLE

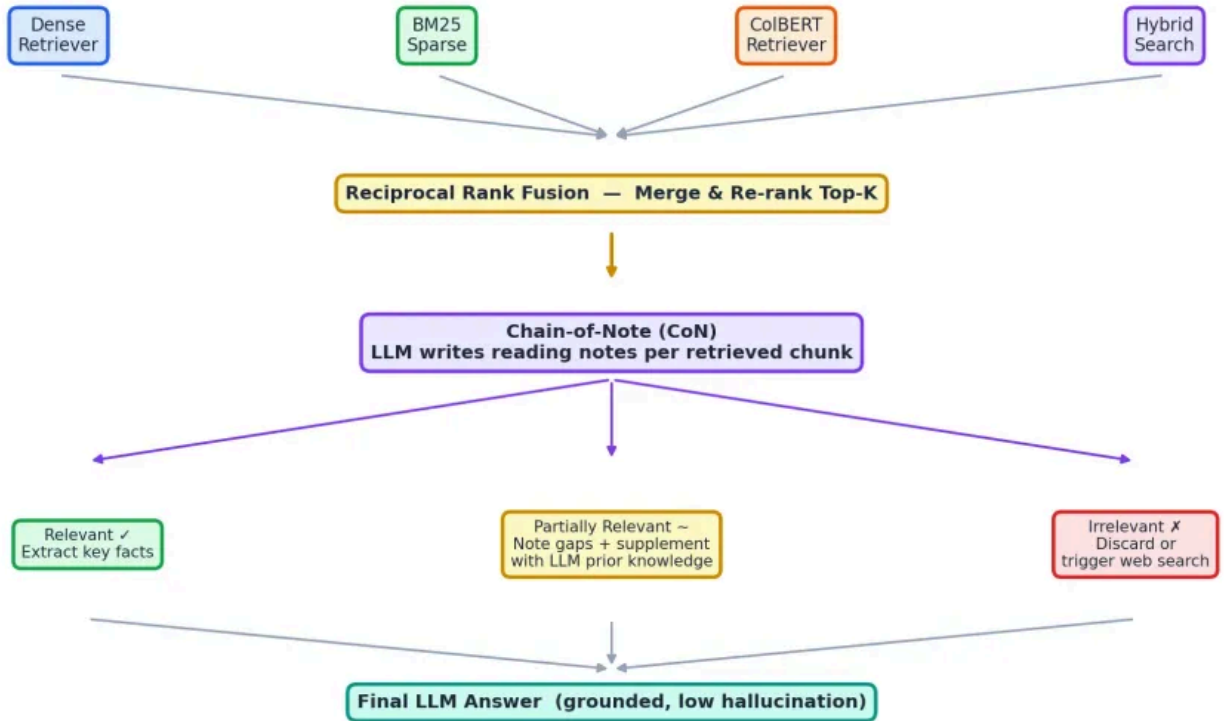
Observability: Log/error messages are token-heavy and exact (`ECONNRESET` , `OOMKilled` , error codes). **ColBERT**'s MaxSim matches those exact tokens where a single dense vector blurs them, so a precise stack-trace symbol still finds the right runbook.

SECURITY EXAMPLE

Security: CVE IDs, rule names, and IOC strings are exact tokens. Dense-only can miss `CVE-2026-1234` under phrasing drift; **ColBERT / DPR** trained on real Q&A preserves the exact-match signal while still catching paraphrased vulnerability descriptions.

6 · Re-ranking & Chain-of-Note Post-processing

Re-ranking & Chain-of-Note Post-processing



Re-ranking with Multiple Retrievers

Don't pick one retrieval method. Run BM25, dense, and ColBERT in parallel, then merge with RRF.

```
from langchain.retrievers import EnsembleRetriever

ensemble = EnsembleRetriever(
    retrievers=[bm25_retriever, dense_retriever, colbert_retriever],
    weights=[0.3, 0.5, 0.2]
)
```

After the ensemble merge, add a **cross-encoder re-ranker**. Cross-encoders process query and document together in one pass — far more accurate than bi-encoder similarity, but too slow across the whole corpus, so run it on the top-K candidates. [cross-encoder/ms-marco-MiniLM-L-6-v2](#) is a solid free option; Cohere Rerank is the easy production choice.

Chain-of-Note (CoN)

One of the most practical techniques here. Instead of dumping retrieved chunks into the context window, make the LLM read each chunk and write notes first.

```
CON_PROMPT = ""
Given the question: {question}
```

Read the following passage and write concise notes:

1. Is this passage relevant to the question?
2. What specific facts from this passage help answer the question?
3. What is missing?

Passage: {passage}

Notes:
""

Three outcomes:

- **Fully relevant** — notes extract the key facts cleanly.
- **Partially relevant** — notes acknowledge gaps; the LLM supplements from its own knowledge without hallucinating.
- **Irrelevant** — the LLM flags it; discard it or trigger a web search.

The hallucination reduction is real: forcing the model to explicitly evaluate each chunk before synthesizing makes it much harder to confabulate. The paper also shows it handles conflicting information across documents much better than vanilla RAG.

OBSERVABILITY EXAMPLE

Observability: Ten runbooks mention "latency"; the cross-encoder surfaces the one for *this* service's dependency, and Chain-of-Note discards a look-alike runbook for a different service — preventing a confidently wrong remediation during an incident.

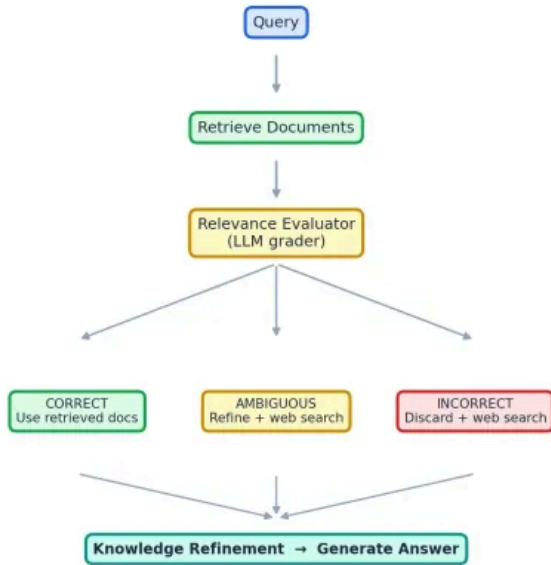
SECURITY EXAMPLE

Security: Chain-of-Note is a guardrail against acting on stale advisories — the note reads "this advisory is for v2, our fleet is v3" and declines rather than raising a false exposure. Re-ranking pushes the exactly-matching control clause above near-duplicates.

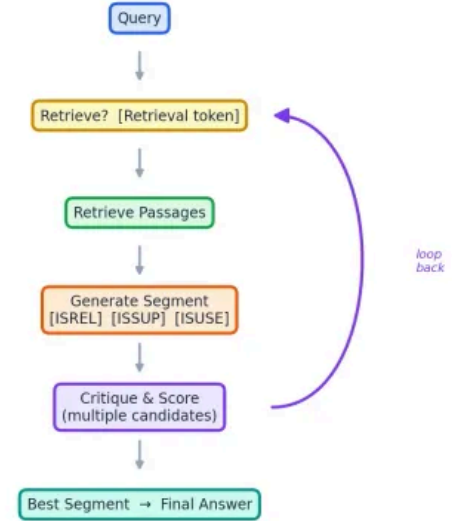
7 · Agentic RAG: CRAG & Self-RAG

Agentic RAG: CRAG & Self-RAG

CRAG — Corrective RAG



Self-RAG



- [Retrieve] — should I fetch more passages?
- [ISREL] — is this passage relevant?
- [ISSUP] — is my claim supported by the passage?
- [ISUSE] — is this output useful overall?

CRAG: Corrective RAG

Add a retrieval evaluator — an LLM grader that scores each retrieved document and decides whether it warrants use. If documents are bad, CRAG refines the query or falls back to web search.

```

from langgraph.graph import StateGraph

def grade_documents(state):
    docs = state["documents"]; question = state["question"]
    scored = [grader_llm.invoke({"doc": d, "question": question}) for d in docs]
    return {
        "documents": [d for d, s in zip(docs, scored) if s.binary_score == "yes"],
        "web_search": any(s.binary_score == "no" for s in scored)
    }

def web_search(state):
    if state["web_search"]:
        state["documents"] += tavily_search.invoke(state["question"])
    return state
  
```

Decision logic: **CORRECT** → proceed to generation; **AMBIGUOUS** → refine query + supplement with web search; **INCORRECT** → discard everything and do a full web search. Big deal where document staleness is a concern — your internal corpus might be six months old; CRAG falls back to fresh web content when local retrieval fails ([arxiv 2401.15884](https://arxiv.org/abs/2401.15884)).

Self-RAG

Trains the LLM to insert special **reflection tokens** during generation, making it explicitly decide whether to retrieve and score its own outputs.

Reflection token	Meaning
[Retrieve]	Should I retrieve more passages?
[ISREL]	Is this passage relevant to the query?
[ISSUP]	Is my claim supported by the passage?
[ISUSE]	Is this output useful overall?

```
# Self-RAG generates multiple candidate segments
# Each scored by [ISREL][ISSUP][ISUSE] tokens
# Best-scored segment selected as output
# Loops back to retrieve more if ISUSE = "no"
```

When to choose: Self-RAG for per-sentence grounding in long-form generation (finer control). CRAG for most production pipelines wanting a corrective mechanism without training a custom model (simpler, more practical).

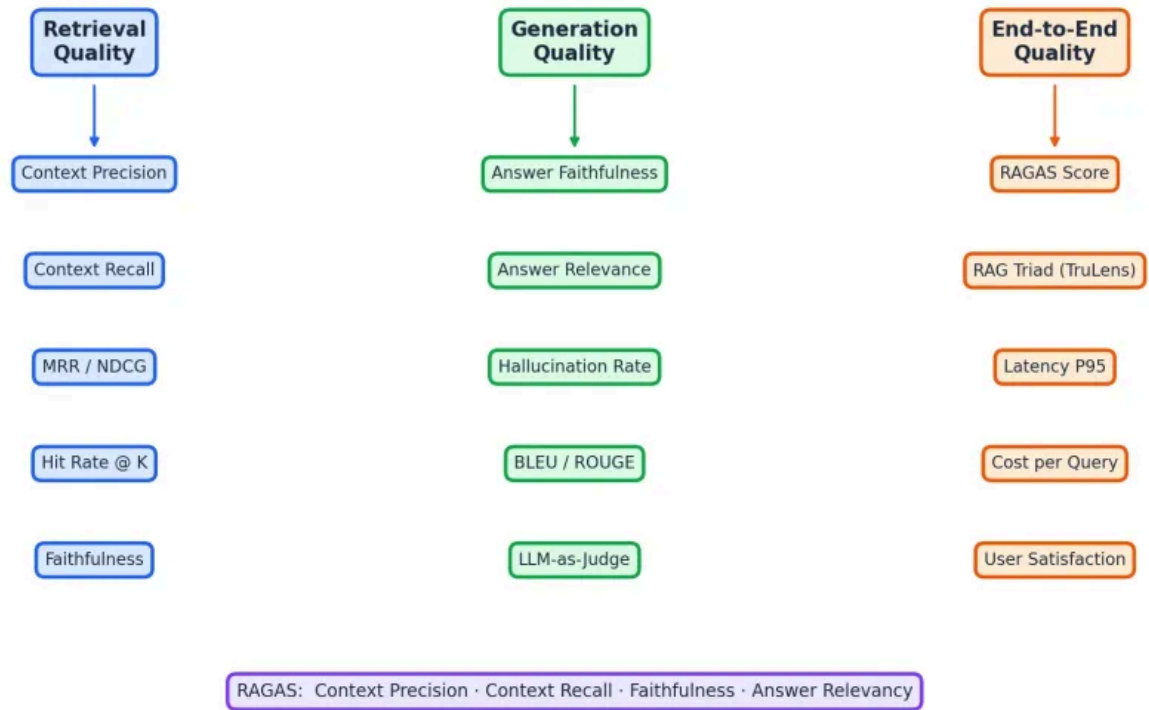
OBSERVABILITY EXAMPLE

Observability: During triage, **CRAG** grades retrieved runbooks; if none match the current alarm signature it doesn't hallucinate a fix — it triggers a fallback search (recent changes, similar traces). **Self-RAG** skips retrieval for a trivial "what does p99 mean?" and retrieves only for environment-specific questions.

SECURITY EXAMPLE

Security: For "is this alert a true positive?", **CRAG** discards low-confidence intel and re-queries authoritative sources before asserting; **Self-RAG's** [ISSUP] means the assistant won't claim an exposure unless retrieved evidence backs it — critical when a false claim triggers real response actions. Pair with human-in-the-loop for consequential actions.

RAG Evaluation Framework



The RAG Triad (TruLens)

Most teams measure end-to-end accuracy and miss where the system is actually failing. The RAG Triad isolates three independent failure modes:

Metric	What it measures	Failure it catches
Context Relevance	Is retrieved context relevant to the query?	Retrieval noise
Groundedness	Is the answer grounded in the context?	Hallucination
Answer Relevance	Does the answer address the query?	Off-topic generation

You can have perfect retrieval and still hallucinate; you can be grounded and still be off-topic. Measuring them separately tells you exactly where to focus.

RAGAS: Automated Pipeline Evaluation

```

from ragas import evaluate
from ragas.metrics import (faithfulness, answer_relevancy,
                           context_precision, context_recall)
from datasets import Dataset

result = evaluate(
    Dataset.from_dict({
        "question": questions, "answer": answers,
        "contexts": contexts, "ground_truth": ground_truths,
    }),
    metrics=[faithfulness, answer_relevancy, context_precision, context_recall],
)

```

Run this on a representative test set **before and after each change**. RAGAS tells you whether a retrieval optimization actually improved generation quality or just looked better in isolation.

What to Track in Production

```

Retrieval:  Hit Rate@K · MRR · NDCG · Context Precision / Recall
Generation: Faithfulness · Answer Relevancy · Hallucination Rate
System:     Latency P50/P95 · Cost-per-query · Throughput

```

LLM-as-Judge

When you don't have ground-truth labels, use a stronger LLM to score outputs (faithfulness 1–5, relevance 1–5, with reasoning as JSON). Not perfect, but surprisingly good for catching obvious failures and cheap to run across large test sets.

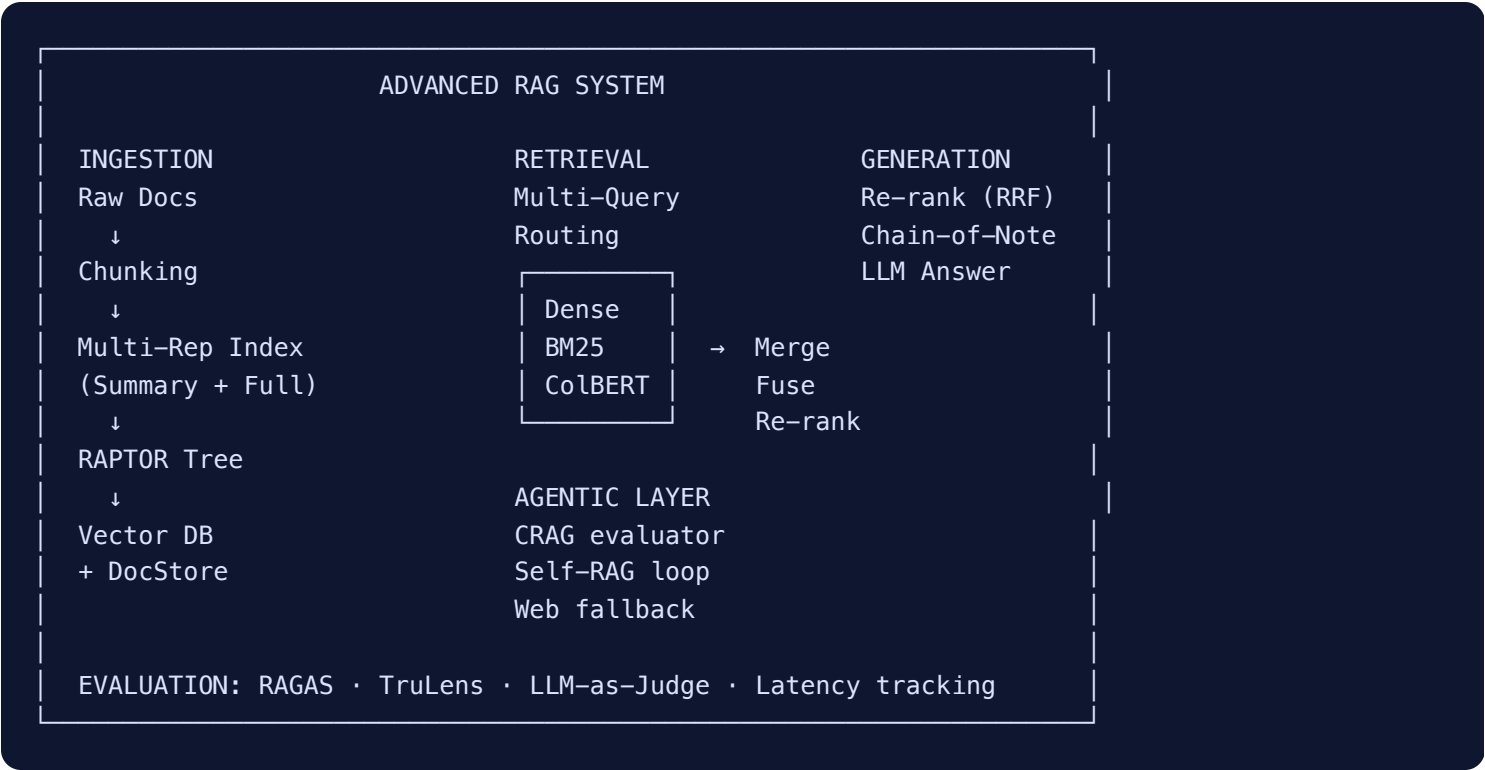
OBSERVABILITY EXAMPLE

Observability: Curate past incidents with known root causes; measure whether the assistant retrieves the correct postmortem (context recall) and proposes the correct remediation (correctness), tracking time-to-answer as a system metric. Regression-gate every index/prompt change so retrieval quality can't silently degrade.

SECURITY EXAMPLE

Security: Groundedness is the headline metric — a security answer must be grounded in retrieved evidence, never invented. Track false-positive/negative rates on a labeled findings set; a drop in groundedness is a release blocker because an ungrounded claim can trigger unwarranted response actions.

9 · Production Architecture Blueprint



Technology Stack

Layer	Options	Recommended
Vector DB	Chroma, FAISS, Pinecone, Weaviate	Pinecone or Weaviate for production
Embedding	OpenAI, Mistral, Cohere, BGE	BGE-M3 or Mistral Embed for cost
Retrieval	Dense, BM25, ColBERT via RAGatouille	Ensemble for best recall
Re-ranker	Cross-encoder, Cohere Rerank	Cohere Rerank for simplicity
Orchestration	LangChain, LlamaIndex, LangGraph	LangGraph for anything agentic
Evaluation	RAGAS, TruLens, DeepEval	RAGAS for automation

Mapping to AgentCore / this platform: orchestration → AgentCore Runtime; agentic layer → the runtime multi-agent patterns; evaluation → the Evaluations component; input/output filtering + access-scoped retrieval → the Guardrails component; shared context → the Memory component.

OBSERVABILITY EXAMPLE

Observability: A read-only assistant — router sends live-metric questions to tools and doc questions through the full retrieve→re-rank→CRAG pipeline; every stage emits a trace span so latency is attributable to retrieval, re-ranking, or generation, and retrieval quality is evaluated continuously.

SECURITY EXAMPLE

Security: Access control is first-class — retrieval is scoped to the indexes/namespaces the requesting identity may see (no cross-tenant/over-privileged data). Output filtering suppresses secrets/PII; consequential actions stay behind human approval.

10 · Implementation Patterns

Pattern 1: Full Advanced RAG Chain (LangChain)

```
from langchain.retrievers.multi_query import MultiQueryRetriever
from langchain.retrievers import EnsembleRetriever, BM25Retriever
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

multi_query = MultiQueryRetriever.from_llm(retriever=dense.as_retriever(), llm=llm)
ensemble = EnsembleRetriever(retrievers=[bm25, multi_query], weights=[0.4, 0.6])

con_prompt = ChatPromptTemplate.from_template("""
Write reading notes for the context, then answer the question.
Context: {context}
Question: {question}
Notes:
Answer:
""")

chain = (
    {"context": ensemble | format_docs, "question": RunnablePassthrough()}
    | con_prompt | llm | StrOutputParser()
)
```

Pattern 2: CRAG with LangGraph

```

from langgraph.graph import END, StateGraph

workflow = StateGraph(GraphState)
workflow.add_node("retrieve", retrieve)
workflow.add_node("grade_documents", grade_documents)
workflow.add_node("generate", generate)
workflow.add_node("web_search_node", web_search)

workflow.set_entry_point("retrieve")
workflow.add_edge("retrieve", "grade_documents")
workflow.add_conditional_edges(
    "grade_documents", decide_to_generate,
    {"web_search": "web_search_node", "generate": "generate"},
)
workflow.add_edge("web_search_node", "generate")
workflow.add_edge("generate", END)
app = workflow.compile()

```

Pattern 3: ColBERT with RAGatouille

```

from ragatouille import RAGPretrainedModel

RAG = RAGPretrainedModel.from_pretrained("colbert-ir/colbertv2.0")
RAG.index(
    collection=[doc.page_content for doc in docs],
    document_ids=[str(i) for i in range(len(docs))],
    index_name="production_index",
    max_document_length=256,
    split_documents=True,
)
results = RAG.search(query="What is the gross revenue of Company X in 2005?", k=10)

```

Decision Tree: Where to Start

```
Is the query simple and well-formed?
├ YES → Standard dense retrieval may suffice
└ NO  → Apply Query Transformation first
      ├── Ambiguous → Multi-Query + RAG-Fusion
      ├── Complex multi-step → Decomposition
      └ Domain mismatch → HyDE / Step-Back

Do you have multiple data sources?
└ YES → Add Query Routing (Logical + Semantic)

Is recall still poor?
└ YES → Upgrade retrieval: DPR → ColBERT → Ensemble + RRF

Are hallucinations still occurring?
└ YES → Add Chain-of-Note post-processing

Is retrieved context unreliable?
└ YES → Add CRAG evaluator + web fallback

Need per-sentence grounding?
└ YES → Self-RAG with reflection tokens

Are you measuring quality?
└ Use RAGAS + TruLens RAG Triad continuously
```

Conclusion

Advanced RAG isn't one technique — it's a layered set of interventions, each targeting a specific failure mode. You don't have to implement everything at once. Start with query transformation (biggest bang for least effort), then layer in better retrieval (ensemble + ColBERT), then Chain-of-Note to cut hallucinations, and only add CRAG or Self-RAG once retrieval itself is solid. Evaluate at every step with RAGAS so you actually know what's improving. The teams that build reliable RAG treat each stage as an independent engineering problem rather than hoping the LLM compensates for poor retrieval. It won't.

References

Resource	Link
RAPTOR Paper	arxiv.org/pdf/2401.18059.pdf
CRAG Paper	arxiv.org/abs/2401.15884
ColBERT Paper	arxiv.org/pdf/2004.12832.pdf
RAGAS	github.com/explodinggradients/ragas

RAGatouille (ColBERT)	github.com/bclavie/RAGatouille
LangGraph CRAG	langgraph CRAG example

Source blog: "Advanced RAG patterns" (AWS Builder Center) — full text and diagrams provided by the user. Notes follow the blog's structure and content; diagrams reproduced to match the originals. Observability & Security example callouts added at explicit request. See [reference/SOURCES.md](#) .